```
import threading
import cv2
import pyttsx3
import RPi.GPIO as GPIO
import time
from ultralytics import YOLO
import queue

# GPIO setup
GPIO.setmode(GPIO.BCM)
TRIG_PIN = 4
ECHO_PIN = 17
GPIO.setup(TRIG_PIN, GPIO.OUT)
GPIO.setup(ECHO_PIN, GPIO.IN)

# YOLO model
yolo = YOLO('yolov8n.pt')

# TTS setup
engine = pyttsx3.init()
engine.setProperty('rate', 160)
engine.setProperty('volume', 1.0)

# Speak dictionary
speak_dct = {
    0: "Person ahead. Please be careful.",
    1: "Bicycle approaching. Stay alert.",
    2: "Car nearby. Be cautious.",
```

3: "Motorcycle detected. Be aware of its movement.",

4: "Airplane overhead. You might hear noise.",

5: "Bus detected. Keep your distance.",

6: "Train nearby. Stay clear of tracks.",

7: "Truck detected. Be careful.",

8: "Boat nearby. Be cautious near water.",

9: "Traffic light ahead. Be mindful of signals.",

10: "Fire hydrant detected. Be careful.",

11: "Stop sign ahead. Please stop.",

12: "Parking meter nearby. Adjust your path.",

13: "Bench detected. You may sit here.",

14: "Bird nearby. Be aware of movement.",

15: "Cat detected. Be cautious of animals.",

16: "Dog nearby. Stay alert for its movement.",

17: "Horse detected. Be cautious of its size.",

18: "Sheep nearby. Be careful.",

19: "Cow detected. Keep your distance.",

20: "Elephant detected. Be very cautious.",

21: "Bear detected. Keep your distance.",

22: "Zebra nearby. Be cautious of movement.",

23: "Giraffe detected. Be mindful of obstacles.",

24: "Backpack detected. Someone may be close by.",

25: "Umbrella detected. Be cautious of its location.",

26: "Handbag detected. Someone may be near.",

27: "Tie detected. Be aware of its location.",

28: "Suitcase detected. Be cautious near luggage.",

29: "Frisbee detected. Be cautious of flying objects.",

30: "Skis detected. Be cautious in snowy areas.", 31: "Snowboard detected. Be aware in snow.",

32: "Sports ball nearby. Be cautious of movement.",

33: "Kite detected. Be aware of objects in the air.",

34: "Baseball bat detected. Be cautious of sports equipment.",

35: "Baseball glove detected. Be aware of sports equipment.",

36: "Skateboard detected. Be cautious of movement.",

37: "Surfboard detected. Be careful near water.",

38: "Tennis racket detected. Be cautious of sports equipment.",

39: "Bottle detected. Be careful of items.",

40: "Wine glass detected. Be cautious of breakable items.",

41: "Cup detected. Be aware of its location.",

42: "Fork detected. Be cautious of sharp objects.",

43: "Knife detected. Be cautious of sharp objects.",

44: "Spoon detected. Be cautious of its location.",

45: "Bowl detected. Be aware of its location.",

46: "Banana detected. Be cautious of slippery surfaces.",

47: "Apple detected. Be aware of food items.",

48: "Sandwich detected. Be aware of food items.",

49: "Orange detected. Be aware of its location.",

50: "Broccoli detected. Be aware of food items.",

51: "Carrot detected. Be aware of its location.",

52: "Hot dog detected. Be aware of food items.",

53: "Pizza detected. Be cautious around food.",

54: "Donut detected. Be aware of its location.",

55: "Cake detected. Be aware of its location.",

56: "Chair detected. Be cautious of obstacles.",

57: "Couch detected. Be cautious of obstacles.",

58: "Potted plant detected. Be aware of obstacles.",

```
    59: "Bed detected. Be cautious of its location.",
    60: "Dining table detected. Be cautious of obstacles.",
    61: "Toilet detected. You are in a restroom.",
    62: "TV detected. Be cautious of furniture.",
    63: "Laptop detected. Be cautious of electronics.",
    64: "Mouse detected. Be aware of small electronics.",
    65: "Remote detected. Be aware of small electronics.",
    66: "Keyboard detected. Be cautious of electronics.",
    67: "Cell phone detected. Be cautious of small electronics.",
    68: "Microwave detected. Be cautious in the kitchen.",
    69: "Oven detected. Be cautious in the kitchen.",
    70: "Toaster detected. Be cautious in the kitchen.",
    71: "Sink detected. You are near a water source.",
    72: "Refrigerator detected. You are in the kitchen.",
    73: "Book detected. Be aware of reading material.",
    74: "Clock detected. You may hear ticking sounds.",
    75: "Vase detected. Be cautious of fragile objects.",
    76: "Scissors detected. Be cautious of sharp objects.",
    77: "Teddy bear detected. A soft object nearby.",
    78: "Hair drier detected. You may hear a loud noise.",
    79: "Toothbrush detected. You are in the bathroom."
}

# Create a queue for speech requests
speech_queue = queue.Queue()

# Object detection announcements
def speak_async(text):
```

```
    #threading.Thread(target=lambda: engine.say(text) or engine.runAndWait()).start()
    speech_queue.put(text)



# Measure distance using ultrasonic sensor
def measure_distance():
    GPIO.output(TRIG_PIN, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(TRIG_PIN, GPIO.LOW)

    start_time = time.time()
    pulse_start = None
    pulse_end = None

    while GPIO.input(ECHO_PIN) == GPIO.LOW:
        pulse_start = time.time()
        if pulse_start - start_time > 0.3:
            return None

    while GPIO.input(ECHO_PIN) == GPIO.HIGH:
        pulse_end = time.time()
        if pulse_end - start_time > 0.3:
            return None

    if pulse_start and pulse_end:
        pulse_duration = pulse_end - pulse_start
        distance = (pulse_duration * 34300) / 2
        return round(distance, 2)
```

```
    else:
        return None


def process_speech_queue():
    while not speech_queue.empty():
        text = speech_queue.get()
        engine.say(text)
        engine.runAndWait()


# Main loop
def main():
    cap = cv2.VideoCapture(0)
    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            frame = cv2.resize(frame, (640, 480))
            distance = measure_distance()
            print(distance)

            print("Object Detection is ON")
            if distance is not None and 10 < distance < 160:
                results = yolo.predict(frame, conf=0.75)
                for box in results[0].boxes:
                    cls = int(box.cls[0])
                    confidence = box.conf[0]
```

```
                print(f"Detected {cls} with confidence {confidence:.2f}")
                if cls in speak_dct:
                    speak_async(speak_dct[cls])



            # Process speech queue
            process_speech_queue()


            #cv2.imshow('Blind Stick View', frame)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

    finally:
        cap.release()
        cv2.destroyAllWindows()
        GPIO.cleanup()

if __name__ == "__main__":
    main()
```